॥ श्रीगणेशाय नमः ॥

Relational Algebra

Queries

Procedural queries
These are formulations of how to retrieve and what data to retrieve from a DB table.

Non procedural queries
These are formulations of what data to retrieve from a DB table (not how to retrieve it).
E.g. Relational calculus [uses FOL, predicate logic]

Relational calculus
- Tuple Relational calculus
  (SQL is similar to this)
  The first version of SQL was developed by IBM.
- Domain Relational calculus
  (QBE: Query by example is similar to this)
  The first version of QBE was developed by Microsoft.

Relational algebra

Basic operators

- π : projection
- ρ : rename
- σ : selection

unary: operate on one table

- $\times$: cross product
- $\cup$: union
- $-$: set difference

binary: operate on 2 tables (or 2 instances of the same table)

Derived operators
These are operators derived from the basic operators.

- $\bowtie$: join (natural, condition) (uses $\times$, σ, π)
- $\cap$: intersection (using $-$)
- $/$: division (using $\times$, π, $-$)

π: Projection
It projects the require attributes from a relation R.

A	B	C	D

σ: selection
It retrieves the records from a relation R that satisfy condition p.

$\sigma_p(R)$

A	B	C
4	6	8
8	5	4
7	3	4
7	3	4

} A relational algebra relation may have duplicate records.

$\pi_{BC}(R)$ →

B	C
6	8
5	4
3	4

} The output of a relational algebra query does NOT contain duplicates.

$\sigma_{A>5}(R)$

A	B	C
8	5	4
7	3	4

no duplicates here either.

Q. Which of these are true?
i. π is commutative
ii. σ is commutative

→ $\pi_{list2}(\pi_{list1}(R)) \stackrel{?}{=} \pi_{list1}(\pi_{list2}(R))$

Say list2 = AB,
list1 = ABC, R = ABCD

In LHS, we'd take ABC then from those we'd take AB

In RHS, From R(ABCD) we get AB
But from AB, we cannot get ABC.

π is not commutative.

$\sigma_{C2}(\sigma_{C1}(R)) \stackrel{?}{=} \sigma_{C1}(\sigma_{C2}(R))$

R = Database of a university.
say C_1 → branch == "CS"
C_2 → marks > 60 %

In LHS,
we'd first select all CS students
then choose CS students with >60% marks

In RHS,
we first get students with >60% marks.
Then from those we choose CS students.

We get the same thing either way.

$\sigma_{C1}(\sigma_{C2}(R)) = \sigma_{C2}(\sigma_{C1}(R)) = \sigma_{C1 \text{ and } C2}(R)$

σ is commutative.

× : cross product
R×S gives a relation gives all attributes of R and S having each record in R paired with each record in S.

R

A	B	C
4	2	5
6	5	8
3	4	5

S

C	D
5	7
7	8

R×S:

A	B	C	C	D
4	2	5	5	7
4	2	5	7	8
6	5	8	5	7
6	5	8	7	8
3	4	5	5	7
3	4	5	7	8

→ The attributes may not be distinct.

If R has x attributes and S has y attributes, then R × S has x+y attributes.
If R has n distinct tuples and S has m distinct tuples, then R × S has n*m tuples.
If R has n tuples and S has 0 tuples then R × S has 0 tuples.
If R × S results in an empty record set, then either R or S has 0 tuples. Both may have 0 tuples as well.

Joins

⋈: natural join

$$R \bowtie S \Leftrightarrow \Pi_{\text{distinct attributes}}\left(\sigma_{\text{equality both common attributes of R and S}}(R \times S)\right)$$

e.g.

A	B	C
a	1	2

×

B	C	D
1	2	d

=

A	B	C	B	C	D
a	1	2	1	2	d

common attributes →

A	B	C	D
a	1	2	d

e.g.

A	B	C
4	2	5
6	5	8
3	4	5

⋈

C	D
5	7
7	8

Note: For performing R ⋈ S here, R.C = S.C (common attributes' not values)

→

A	B	C
4	2	5
6	5	8
3	4	5

C	D
5	7
7	8

step 1: cross product →

A	B	C	C	D
4	2	5	5	7
4	2	5	7	8
6	5	8	5	7
6	5	8	7	8
3	4	5	5	7
3	4	5	7	8

step 2: equality on common attributes →

A	B	C	C	D
4	2	5	5	7
3	4	5	5	7

step 3: project distinct attributes →

A	B	C	D
4	2	5	7
3	4	5	7

If there are no common attributes in R and S, R ⋈ S = R × S.

Conditional join

$$R \bowtie_c S = \sigma_c(R \times S)$$

- allows duplicate attributes (cross product)

×: repetition ✓ — R(ABCD) × S(BCD) will have ABCBCD

⋈: repetition ✗ — ⋈ ABCD

⋈c: repetition ✓ — ⋈c ABCBCD

Q. R

A	B	C
4	2	5
6	5	8
3	4	5

S

C	D
5	7
7	8

$R \bowtie_{R.C>S.C} S = ?$

→ $R \bowtie_{R.C>S.C} S = \sigma_{R.C>S.C}(R \times S)$

R.C > S.C

R × S =

A	B	C	C	D
4	2	5	5	7
4	2	5	7	8
6	5	8	5	7
6	5	8	7	8
3	4	5	5	7
3	4	5	7	8

output:

A	B	C	C	D
6	5	8	5	7
6	5	8	7	8

$R \bowtie S = ?$ (when condition is not given, natural join)

→ $\Pi_{\text{distinct attributes}}\left(\sigma_{\text{common attributes equality}}(R \times S)\right)$

R × S =

A	B	C	C	D
4	2	5	5	7
4	2	5	7	8
6	5	8	5	7
6	5	8	7	8
3	4	5	5	7
3	4	5	7	8

output:

A	B	C	D
4	2	5	7
3	4	5	7

Outer joins

⟕ : Left outer join

$R ⟕ S = R ⋈ S \cup$ (Records of R that fail the join condition)

e.g.

R

A	B	C
4	2	5
6	5	8
3	4	5

S

C	D
5	7
7	8

→ $R ⋈ S$

A	B	C	D
4	2	5	7
3	4	5	7

$R ⟕ S$

A	B	C	D
4	2	5	7
3	4	5	7
6	5	8	NULL

not in $R ⋈ S$

⟖ : Right outer join

$R ⟖ S = R ⋈ S \cup$ (Records of S that fail the join condition)

e.g.

R

A	B	C
4	2	5
6	5	8
3	4	5

S

C	D
5	7
7	8

$R ⋈ S$

A	B	C	D
4	2	5	7
3	4	5	7

$R ⟖ S$

A	B	C	D
4	2	5	7
3	4	5	7
NULL	NULL	7	8

not in $R ⋈ S$

⟗ : Full outer join

$R ⟗ S$ = left outer join $\cup$ right outer join

$R ⟗ S$

A	B	C	D
4	2	5	7
3	4	5	7
6	5	8	NULL
NULL	NULL	7	8

Conditional left join

- duplicates are allowed.

$R ⟕_C S = R ⋈_C S \cup$ (rows of R that fail the join condition)

R

A	B	C
4	2	5
6	5	8
3	4	5

S

C	D
5	7
7	8

$R ⟕_{R.C>S.C} S$ — Repeated attributes (conditional join)

A	B	C	C	D
6	5	8	5	7
6	5	8	7	8
4	2	5	N	N
3	4	5	N	N

Conditional right join

$R ⟖_C S = R ⋈_C S \cup$ (rows of S that fail the join condition)

R

A	B	C
4	2	5
6	5	8
3	4	5

S

C	D
5	7
7	8

$R ⟖_{R.C<S.C} S$

A	B	C	C	D
4	2	5	7	8
3	4	5	7	8
N	N	N	5	7

Conditional full outer join

Join queries

(when 2 tables are needed to be joined)

$R(A, \ldots)$, $S(B, \ldots)$

Retrieve A values of R that are more than some B values of S. (some = any, at least 1)

e.g.

R

A
10
20 ✓
30 ✓
40 ✓

S

B
15
25
40

→ Output:

A
20
30
40

$$\Pi_A\left(\sigma_{R.A>S.B}(R \times S)\right)$$

OR

$$\Pi_A\left(R \underset{R.A>S.B}{\bowtie} S\right)$$

{ Some, any, atleast with $>, <, \geq, \leq$ } → conditional join OR cross product followed by selection

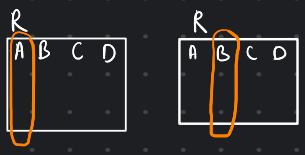
{ some, any, at least with $=$ } → natural join (not always) (if the condition is on common attributes)

ρ : Rename

It is used to rename tables or attributes.
If we compare R with S, then there's no need of renaming.
If we compare R with R, then renaming is needed.

$R(A, B, C, D)$

$\sigma_{R.A>R.B}(R \times R)$



To avoid ambiguity, we need renaming.

We can rename tables or attributes or both.

Student (sid, sname, age)

ρ (temp, stud) ⇒ temp (sid, sname, age) } renaming a table.

$\rho_{I,N,A}$ (student) ⇒ student (I, N, A) } renaming all attributes.

$\rho_{\substack{sid \to I \\ age \to A}}$ (student) ⇒ student (I, sname, A) } renaming selective attributes.

Q. Enroll (sid, cid), student (sid, age). $\Pi_{student.sid}\left(\sigma_{student.sid = enroll.sid}(student \times enroll)\right)$

→ students who are enrolled in some course.

other way: $\Pi_{student.sid}(student \bowtie enroll)$

Q. Emp (eid, salary, gender). Retrive female emp ids, whose salary > salary of some male employee.

→ We need 2 tables, one for females and one for males.

$$\Pi_{eid}\left(\sigma_{gen=female}(Emp) \underset{salary > S}{\bowtie} \rho_{I,S,G}\left(\sigma_{gen=male}(Emp)\right)\right)$$

We rename the male attributes to I, S, G
The female table attributes are eid, salary, gender.

OR

$$\Pi_{eid}\left(\sigma_{\substack{salary \\ > \\ S}}\left(\sigma_{\substack{gender \\ =female}}(Emp) \times \left(\rho_{I,S,G}\left(\sigma_{\substack{gender \\ =male}}(Emp)\right)\right)\right)\right)$$

When we have the same table, we should always rename.

Q. Emp(eid, sal, dno). Retrieve eids whose sal > some employees of dno = 5.

→

$$\Pi_{eid}\left(Emp \underset{sal > S}{\bowtie} \left(\rho_{E,S,d}\left(\sigma_{dno=5}(Emp)\right)\right)\right)$$

OR using cross product, $\Pi_{eid}\left(\sigma_{sal>S}\left(Emp \times \left(\rho_{e,s,d}\left(\sigma_{dno=5}(Emp)\right)\right)\right)\right)$

Q. Stud(sid, age). Dept(did, age). Retrieve sids of students whose age > age of some dept.

→

$$\Pi_{sid}\left(Stud \underset{\substack{Stud.age \\ > \\ Dept.age}}{\bowtie} Dept\right)$$

$R(A, \ldots)$, $S(B, \ldots)$

Retrieve 'A' values of R that are greater than every 'B' value of S.

{every} → complementation.

Q.

R

A
10
20
20
40

S

B
15
25
30

Output:

A
40

$X \equiv$ A values of R > every B value of S.

$\bar{X} \equiv$ A values of R ≤ some B values of S. ← so complement of this is the reqd. result.

$X = \cup - \bar{X} =$ {All possible values of A} − {A values ≤ some B value}

$$\Pi_A(R) - \Pi_A\left(R \underset{R.A \le S.B}{\bowtie} S\right)$$

Q. Emp(eid, dno, sal). Retrieve eids whose salary is more than every emp. of dept # 5.

→

= ∪ (eids whose sal ≤ some emp in dept # 5)

$$\Pi_{eid}(Emp) - \Pi_{eid}\left(Emp \underset{\substack{Emp.sal \\ \le \\ five.sal}}{\bowtie} \rho(five, \sigma_{dept=5}(Emp))\right)$$

Q. Emp(eid, sal, gender). Retrieve eids of female employees whose sal > every male employees' salaries.

→

= ∪ − (fem. with sal ≤ some guy's sal)

$$= \Pi_{eid}\left(\sigma_{\substack{gender \\ = female}}(Emp)\right) - \Pi_{eid}\left(\sigma_{\substack{gender \\ = female}}(Emp) \underset{\substack{Emp.sal \\ \le \\ S}}{\bowtie} \rho_{E,S,G}\left(\sigma_{\substack{gender \\ = male}}(Emp)\right)\right)$$

Q. Emp(eid, sal). Retrieve eids with the max. salary.

→ = sal of emp ≥ every other sal

= ∪(sal of emp < some other sal)

$$\Pi_{eid}(Emp) - \Pi_{eid}\left(Emp \underset{sal<S}{\bowtie} \rho_{I,S}(Emp)\right)$$

So max A value in $R(A, \ldots)$

= {value of A ≥ every other A value}

= ∪{value of A < some value of A}

$$= \Pi_A(R) - \left(R \underset{\substack{R.A \\ < \\ S.A}}{\bowtie} \rho(S, R)\right)$$

Similarly we can get the minimum.

Q. eid whose sal ≤ every other sal

→ = ∪(eid whose sal > some other sal)

$$\Pi_{eid}(Emp) - \Pi_{eid}\left(Emp \underset{sal>S}{\bowtie} \rho_{1,s}(Emp)\right)$$

Q. Emp(Eid, sal, dno) · Eids who get max salary for each dept.

→ X = {eid whose sal ≥ sal of every emp of same dept}

= ∪ − {eids whose sal < sal of every emp of same dept}

$$\Pi_{eid,dno}(Emp) - \Pi_{\substack{eid,\\ dno}}\left(Emp \underset{\substack{sal<S\\ and\\ dno=d}}{\bowtie} \rho_{2,S,D}(Emp)\right)$$

Q. Family (parent, child, dob)
Retrieve the set of all "child" attributes greater than or equal to every "child"'s dob of the same parent.

→ Essentially we need the youngest children for each parent.

Family

parent	child	dob	
A	B	1995	
A	C	1998	✓
A	D	1996	
E	F	1990	
E	G	1996	✓

$$\Pi_{child}(Family) - \Pi_{child}\left(Family \underset{\substack{dob<d\\ and\\ parent=p}}{\bowtie} \rho_{p,c,d}(Family)\right)$$

/ or % : Division

Enroll

sid	cid
s1	c1
s1	c2
s1	c3
s2	c1
s2	c3
s3	c1

course

cid
c1
c2
c3

$\pi_{sid,cid}(Enroll) \,/\, \pi_{cid}(course)$ = sids present with every cid

sid
s1

$\frac{sid, cid}{cid} = sid$

Attributes in the result are those in the numerator but not in the denominator.
(and those values which are present for every value of the denominator attributes).

Explanation

$\pi_{sid,cid}(E) \,/\, \pi_{cid}(C)$ ⇒ sids enrolled with every course in C.

E

sid	cid
1	1
1	2
1	3
2	1
2	3
3	1
3	2

C

cid
1
2
3

Result

sid
1

step 1: sids not enrolled in every course
= sids enrolled in proper subset of all courses.

$\underbrace{\pi_{sid}(E) \times \pi_{cid}(C)}_{\text{all possible enrollments of students (all possible sid, cid pairs)}} - \pi_{sid,cid}(E)$

⇓ π_{sid}

students not enrolled in all of the courses

step 2: students enrolled in every course = students enrolled in some course (U) − students not enrolled in all courses. (result of step 1)

= division

E

sid	cid
s1	c1
s1	c2
s1	c3
s2	c1
s2	c2
s3	c1

C

cid
c1
c2
c3

$\pi_{sid}(E) \times \pi_{cid}(C)$

sid	cid
s1	c1
s1	c2
s1	c3
s2	c1
s2	c2
s2	c3
s3	c1
s3	c2
s3	c3

−

sid	cid
s1	c1
s1	c2
s1	c3
s2	c1
s2	c2
s3	c1

≡

sid	cid
s2	c3
s3	c2
s3	c3

π_{sid} ⇒

sid
s2
s3

students NOT enrolled in ALL courses.

sid
s1
s2
s3

−

sid
s2
s3

⇒

sid
s1

Division can be given as

$$\pi_{AB}(R) / \pi_{B}(S) = \pi_{A}(R) - \pi_{A}\left(\pi_{A}(R) \times \pi_{B}(S) - \pi_{AB}(R)\right)$$

Q. R

A	B	C	D
a_1	b_1	c_1	d_1
a_1	b_1	c_2	d_2
a_1	b_2	c_1	d_3

S

C	D
c_1	d_1
c_2	d_2

$\pi_{ABCD}(R) / \pi_{CD}(R)$

$\rightarrow = \pi_{AB}(R) - \pi_{AB}\left(\pi_{AB}(R) \times \pi_{CD}(S) - \pi_{ABCD}(R)\right) =$

A	B
a_1	b_1
a_2	b_2

Q. Enrolled (sid, cid), course (cid, instructor). Retrieve sids of students enrolled in some course taught by Korth.

→

$$\pi_{sid}\left(\sigma_{Enroll.cid = course.cid}\left(Enroll \times \sigma_{instructor = Korth}(course)\right)\right)$$

OR $\pi_{sid}\left(Enroll \bowtie \sigma_{instructor = Korth}(course)\right)$

Q. Retrieve sids of students enrolled for every course taught by Korth.

→

$$\pi_{sid, cid}(Enroll) / \pi_{cid}\left(\sigma_{instructor = Korth}(course)\right)$$

OR $\pi_{sid}(Enroll) - \pi_{sid}\left(\pi_{sid}(Enroll) \times \pi_{cid}\left(\sigma_{inst = Korth} course\right) - \pi_{sid, cid} Enroll\right)$

- $\pi_{sid}(Enroll)$: all students
- $\pi_{sid}(Enroll) \times \pi_{cid}(\sigma_{inst = Korth} course)$: all possible sid, cid for Korth students
- $\pi_{sid, cid} Enroll$: all Korth enrollments
- $\pi_{sid}(\pi_{sid}(Enroll) \times \pi_{cid}(\sigma_{inst = Korth} course) - \pi_{sid, cid} Enroll)$: students not in all Korth courses
- $\pi_{sid}(Enroll) - \pi_{sid}(\pi_{sid}(Enroll) \times \pi_{cid}(\sigma_{inst = Korth} course) - \pi_{sid, cid} Enroll)$: students in all Korth courses.

Q. WorksFor (eid, pid), Project (pid, pname). Find eids who work for some database project.

→

$$\pi_{eid}(WorksFor \bowtie \sigma_{pname = database}(Project))$$

Q. who works for every db proj

→

$$\pi_{eid, pid}(WorksFor) / \pi_{pid}\left(\sigma_{pname = database}(project)\right)$$

Set operators: union (∪), intersection (∩), and set difference (-)

↳ derived

To use these, R and S must be union compatible.

R and S are union compatible iff:
- Arity of R = arity of S (arity = # of attributes i.e. degree).
- Domain of each attribute of R must be same as that of the attributes of S.

The schema for R∪S, R∩S, and R-S is the same as the schema for R (not S).

The resultant result sets have no duplicate records.

Q. R

A	B	C
2	4	6
3	5	7
3	5	7
4	5	6

S

D	E	F
2	4	6
2	4	6
4	5	6
3	5	9

→ The tables may have duplicates.

R∪S

A	B	C
2	4	6
3	5	7
4	5	6
3	5	9

R∩S

A	B	C
2	4	6
4	5	6

R-S

A	B	C
3	5	7

→ Results do NOT contain duplicates

∩ is a derived operator. $R \cap S = R - (R - S)$

$R(A, B)$, $S(A, B)$

If the attribute names are the same,

① $R \cap S = R - (R - S)$

② $R \cap S = R \bowtie S$

③ $R \cup S = R \boxplus S$

→ say R

A	B
4	6
2	5

S

A	B
4	6
2	8

$R \cap S = R \bowtie S =$

A	B
4	6

$R \boxplus S$

A	B	
4	6	→ common attr. equal
2	5	→ on left only
2	8	→ on right only

$R(A, B)$, $S(C, D)$

If attribute names are not same,

① $R \cap S = R - (R - S)$

② $R \bowtie S \neq R \cap S$

③ $R \cap S = R \bowtie \rho_{\substack{C \to A \\ D \to B}}(S)$ → make attr. names same

④ $R \cup S \neq R \boxplus S$

⑤ $R \cup S = R \boxplus \rho_{\substack{C \to A \\ D \to B}}(S)$

Queries by using set operations

Common data:

Student

sid	sname	age
s1	A	-
s2	B	-
s3	C	-
s4	D	-

Course

cid	cname	instr
c1	DB	Korth
c2	DB	Korth
c3	DB	Navathe
c4	CD	Navathe
c5	OS	Galvin

Enroll

sid	cid	fee
s1	c1	
s1	c2	
s1	c3	
s1	c4	
s2	c1	
s2	c2	
s1	c5	
s3	c3	
s4	c5	

Q. Retrieve sids enrolled for some course taught by Korth or some course taught by Navathe.

$\pi_{sid}(Enroll \bowtie \sigma_{instr = Korth \vee instr = Navathe}(course))$ OR $\pi_{sid}(Enroll \bowtie \sigma_{instr = Korth}(course)) \cup \pi_{sid}(Enroll \bowtie \sigma_{instr = Navathe}(course))$

Q. Retrieve sids enrolled for some course taught by Korth and some course taught by Navathe.

$\pi_{sid}(Enroll \bowtie \sigma_{instr = Korth}(course)) \cap \pi_{sid}(Enroll \bowtie \sigma_{instr = Navathe}(course))$

Q. Retrieve sids of students enrolled only for courses taught by Korth.

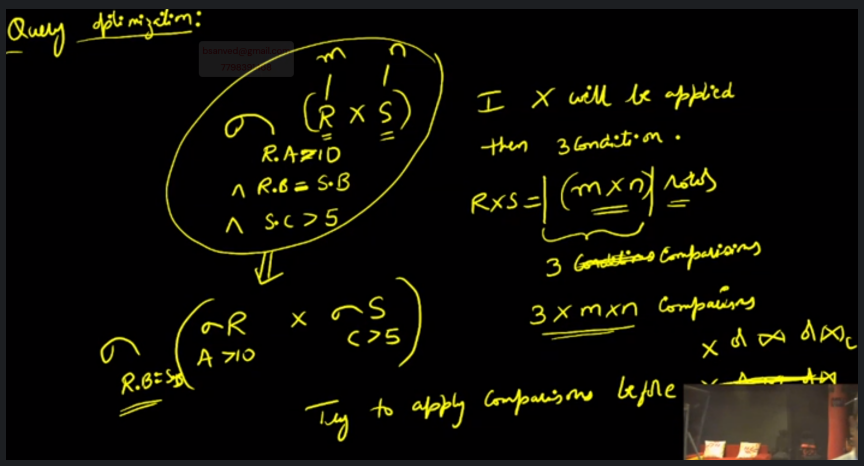
→ = Total − {students in some course by someone other than Korth}.

$\pi_{sid}(Enroll) - \pi_{sid}(Enroll \bowtie \sigma_{instr \neq Korth}(course))$

Q. Retrieve sids of students enrolled for some course taught by Korth.

$\pi_{sid}(Enroll \bowtie \sigma_{instr = Korth}(course))$

Query optimization



e.g. $\sigma_{R.A>10}(R \bowtie S) \Leftrightarrow \sigma_{A>10}(R) \bowtie S$

e.g. $\sigma_{(R.A>10) \wedge (S.C>5)}(R \bowtie S) \Leftrightarrow \sigma_{A>10}(R) \bowtie \sigma_{C>5}(S)$

↳ lesser # of rows to join

e.g. Enroll(sid, cid), Course(cid, instructor).
Enroll has 10000 rows, course has 100 rows.

$\sigma_{instructor = Korth}(Enroll \bowtie course)$

$\Leftrightarrow \sigma_{instructor = Korth}(\sigma_{Enroll.cid = course.cid}(Enroll \times course))$

$Enroll \times course$: 10,00,000 rows
(20,00,000 comparisons for 2 conditions)

This can be satisfied as:

$\sigma_{Enroll.cid = course.cid}(Enroll \times \sigma_{instructor = Korth}(course))$

- Enroll: 10,000 rows
- $\sigma_{instructor = Korth}(course)$: ↳ 100 comparisons, ≤100 rows (say 5)
- $Enroll \times \sigma_{instructor = Korth}(course)$: 50,000 rows
  ↳ 50,000 comparisons (assuming 5 rows by Korth)

e.g. Emp(eid, gender, sal)

$\Pi_{eid}(Emp \bowtie_{(gen=fem) \wedge (G=male) \wedge (sal>S)} \rho_{1,G,S}(Emp))$

Female emps with sal > atleast 1 male emp's sal

↳ $\sigma_{gen=fem}(Emp) \bowtie_{sal>S} \sigma_{G=male}(\rho_{1,G,S}(Emp))$

Queries related to atleast / only / atmost:

Something like sids of students who took atleast 2 courses.

Q. Enroll(sid, cid). Retrieve sid's of students who've enrolled for atleast 2 courses.

→

sid	cid
s1	c1
s1	c2
s1	c3
s2	c1
s2	c2
s3	c1

⋈

sid	cid
s1	c1
s1	c2
s1	c3
s2	c1
s2	c2
s3	c1

This row in the join would indicate that the same student s_1 is enrolled in different courses c_1 and c_2.

→ A row of the join would be like

sid	cid	sid	cid

say a row of the join is

s_1	c_1	s_1	c_2

If we get $Enroll1.sid = Enroll2.sid$
and $Enroll1.cid \neq Enroll2.cid$

Meaning there's 1 sid with 2 cids.

$$\rho(T_1, Enroll) \bowtie \rho(T_2, Enroll)$$
$$T_1.sid = T_2.sid \wedge T_1.cid \neq T_2.cid$$

Q. Enroll (sid, cid). Get sid's of students in atleast 3 courses.

→ T_1 T_2 T_3 → renamed Enroll tables

$$\sigma_{T_1.sid = T_2.sid = T_3.sid \wedge T_1.cid \neq T_2.cid \neq T_3.cid} (T_1 \times T_2 \times T_3)$$

only → a chance of subtraction.

Sid's enrolled in only 1 course = Sid's in atleast 1 course - Sid's in atleast 2 courses.

$$\underbrace{\pi_{sid}(Enroll)}_{\text{atleast 1 course}} - \underbrace{\pi_{sid}(Enroll \bowtie_{sid=S \wedge cid \neq c} \rho_{S,C}\, Enroll)}_{\text{atleast 2 courses}} \equiv \text{exactly 1 courses}$$

Q. sids in atleast 10 courses.

→ we'd need to perform a cross product on 10 tables.

In SQL, we can just write it in a simple way due to features like GROUP BY, HAVING, etc.

Q. sids enrolled in only 2 courses.

→ exactly 2 = atleast 2 − atleast 3

$$= \pi_{sid}(\rho(E_1, Enroll) \bowtie_{E_1.sid = E_2.sid \wedge E_1.cid \neq E_2.cid} \rho(E_2, Enroll)) - \pi_{sid}(\sigma_{E_1.sid = E_2.sid = E_3.sid \wedge E_1.cid \neq E_2.cid \neq E_3.cid}(\rho(E_1, Enroll) \times \rho(E_2, En) \times \rho(E_3, L)))$$

Q. sids in atmost 1 course.

Atmost 1 = All − Atleast 2

$\leq n$ = Total − $> n$ = Total − $\geq (n+1)$

Stud (sid, sname)

S_1
S_2
S_3
S_4

Enroll (sid, cid)

S_1	C_1
S_1	C_2
S_1	C_3
S_2	C_1
S_2	C_2
S_3	C_1

Result

sid
S_3
S_4

$= \Pi_{sid}(Stud) - \left[\text{sids in atleast 2 courses}\right]$

NOTE: get total from Stud, not Enroll so as not to miss S_4.

Q. sids in atmost 2 courses.

— {all students} − {in atleast 3 courses}

$= \Pi_{sid}(Stud)$ − {atleast 3 courses}

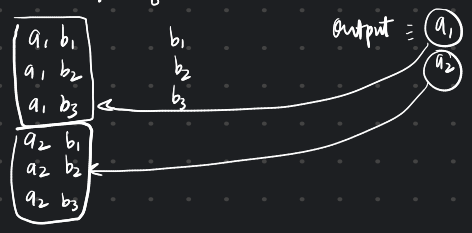
↳ not from Enroll

Some theory related to division:

A relation R has X set of attributes and n distinct tuples. A relation S has Y set of attributes and m distinct tuples.

1. Condition required for R / S is X ⊃ Y.
2. Resulting attribute set of R / S is X-Y.
3. Cardinality of R / S: minimum = 0, maximum = floor(n/m).

$\Pi_{AB}(R) / \Pi_{B}(S)$



$\left\lfloor \frac{n}{m} \right\rfloor = \left\lfloor \frac{6}{3} \right\rfloor = 2$

Relation R with n distinct tuples, S with m distinct tuples.

Relational algebra expression	Cardinality (min, max)	Commutative	
$\Pi_A(R)$	$(1, n)$ ↑ all values are same; ↑ all values are different	×	
$\sigma_c(R)$	$(0, n)$	✓	$\sigma_{c1}(\sigma_{c2}(\cdot)) = \sigma_{c2}(\sigma_{c1}(\cdot))$
$R \times S$	$(n \times m)$	✓	
$R \bowtie S$ $R \bowtie_c S$	$(0, n \times m)$ ↑ no row satisfying condition	✓	
$R \bowtie S$	$(n, n*m)$	×	
$R \bowtie S$	$(m, n*m)$	×	
$R \bowtie S$	$(\underbrace{\max(n,m)}_{\text{min value}}, m*n)$	✓	
$R \cup S$	$(\underbrace{\max(n,m)}_{\text{(one is subset of other)}}, \underbrace{m+n}_{\text{(disjoint sets)}})$	✓	
$R \cap S$	$(0, \min(n,m))$	✓	
$R - S$	$(0, n)$	×	
R / S	$(0, \lfloor \frac{n}{m} \rfloor)$	×	

Q. Stud (sid, sname). Enroll (sid, cid) How many (min, max) tuples in (Stud ⋈ Enroll) ?

100 tuples 50 tuples

sid : PRIMARY KEY (sid, cid) : PRIMARY KEY

→

Key

sid	sname
S_1	A
S_2	A
S_3	B
S_4	B
S_5	A

sid	cid
S_1	C_1
S_1	C_2
S_1	C_3
S_2	C_7

All rows participate in a natural join.

max = 50

min = No rows match.

= 0

If sid is foreign key,

all rows in Enroll would match (sids) with those in Stud.

So

min = |Enroll| = 50